J-CO, A Framework for Fuzzy Querying Collections of *JSON* Documents (Demo)

Paolo Fosci and Giuseppe Psaila(✉)

University of Bergamo, Viale Marconi 5, 24044 Dalmine, BG, Italy
{paolo.fosci,giuseppe.psaila}@unibg.it
http://www.unibg.it

Abstract. This paper accompanies a live demo during which we will show the *J-CO Framework*, a novel framework to manage large collections of *JSON* documents stored in *NoSQL* databases. *J-CO-QL* is the query language around which the framework is built; we show how it is able to perform fuzzy queries on *JSON* documents.

This paper briefly introduces the framework and the cross-analysis process presented during the live demo at the conference.

Keywords: Collections of JSON documents · Framework for managing JSON data sets · Fuzzy querying · Live demo.

1 Introduction

In the era of Big Data, classical relational technology has shown its limitations. In fact, apart from very large volumes of data, the main characteristic of them is their *variety*, both in terms of content and in terms of structure [11]. In order to easily represent complex data, *JSON* (JavaScript Object Notation) has become the *de-facto* standard for sharing and publishing data sets over the Internet. In fact, it is able to represent complex objects/documents, with nesting and arrays; its simple syntactic structure makes it possible to easily generate and parse *JSON* data sets within procedural and object-oriented programming languages.

Straightforwardly, *NoSQL* DBMSs [7] for managing *JSON* documents have been developed and have become very popular. They are called *JSON Document Stores*, since they are able to natively store *JSON* documents in a totally schema-free way. The most popular one is *MongoDB*, but some others, which are attracting more and more users, are *AWS DocumentDB* and *CouchDB*.

However, although these systems are quite effective in storing very large collections of *JSON* documents, there is not a standard query language that allows for retrieving and manipulating data: each system has its own query language. Furthermore, they do not give a unified view of document stores: if data are provided by various JSON stores, they must be managed separately, by exporting data from one document store, to import them into another one.

These considerations motivated the development of the *J-CO Framework* (where *J-CO* stands for *J*SON *CO*llections). The goals we targeted during its development are manifold:

T. Andreasen et al. (Eds.): FQAS 2021, LNAI 12871, pp. 142–153, 2021.
https://doi.org/10.1007/978-3-030-86967-0_11

- we wanted a framework able to retrieve data from and store data to any *JSON Document Store*, independently of its query language;
- we wanted to provide a unique tool, provided with a non-procedural query language, able to integrate and transform collections of *JSON* documents;
- we wanted to provide native support to spatial operations, to cope with very frequent geo-tagging of *JSON* documents, such as in *GeoJSON* documents.

Furthermore, the *J-CO Framework* aims at being a constantly evolving laboratory project, in which to address novel problems concerning the management of *JSON* documents that can be solved either by defining new statements in the query language, or by extending the underlying data model, so as to support new features, or both. For example, we are currently extending the data model to support fuzzy sets [12], to deal with various forms of uncertainty and imprecision that can affect data (a first step is presented in [10] and applied in [6]).

The development of the *J-CO Framework* started at the end of 2016 stimulated by our participation to the *Urban Nexus* project [4]. Funded by the University of Bergamo, in this project computer scientists and geographers worked together to identify a data driven approach to study mobility of city users, but integrating many (possibly big) data sources, including data published in Open Data portals. The need for providing analysts with a tool to easily integrate and analyze so heterogeneous data sets gave the idea to develop the *J-CO Framework*. Since its birth, [2], the framework has significantly evolved, as far as both the query language [1,3] and the components of the framework [8] are concerned.

During the live demo, we showed how the *J-CO Framework* easily works on *JSON* data by exploiting fuzzy sets. In this accompanying paper, Sect. 2 briefly presents the *J-CO Framework*; Sect. 3 briefly describes the features of its query language; Sect. 4 shows a sample query (presented during the live demo too) that illustrates its capability. Finally, Sect. 5 draws the conclusions.

2 The *J-CO Framework*

The *J-CO Framework* is designed to provide analysts and data scientists with a powerful tool to integrate, transform and query data represented as *JSON* documents collected in various *JSON* Stores. It is based on three distinct layers, depicted in Fig. 1.

- *Data Layer.* In principle, this layer encompasses any *JSON Document Store*. Currently, we encompass *MongoDB*, *ElasticSearch* and *J-CO-DS*, a lightweight *JSON* store developed within the *J-CO* project, designed to be able to manage very large *JSON* documents, such as *GeoJSON* layers [5].
- *Engine Layer.* This layer encompasses the *J-CO-QL Engine*, i.e., the software tool that processes collections of *JSON* documents retrieved from *JSON* stores, by executing queries written in the *J-CO-QL* language; results are stored again into some *JSON* store.
- *Interface Layer.* This layer encompasses *J-CO-UI*, the user interface of the framework. It provides functionalities to write queries step-by-step, possibly rolling back queries, as well as to inspect intermediate results of the process.

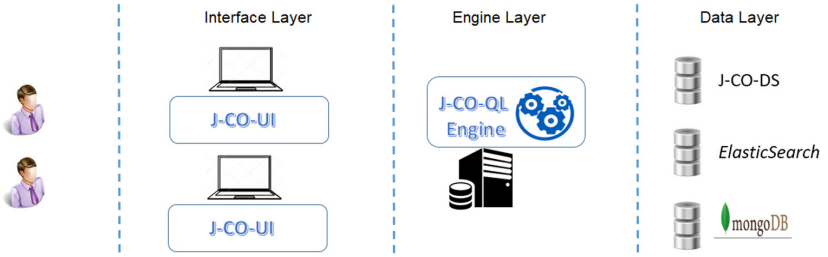


Fig. 1. Layers for the *J-CO Framework*.

The *J-CO-QL Engine* and the *J-CO-UI* are independent applications, so that they can be executed on different machines: for example, the *J-CO-QL Engine* on a powerful cloud virtual machine, while the *J-CO-UI* on the PC of the analyst. The *J-CO-QL Engine* can access any *JSON Document Store* through a standard internet connection. Thus, *J-CO* is actually a platform-independent framework: this choice allows for further extending the framework in the future, for example by providing a parallel version of the engine possibly based on the Map-Reduce paradigm. The reader can find a detailed description of the *J-CO Framework* in [9].

3 The Query Language: *J-CO-QL*

J-CO-QL is the query language around which the framework is built. We do not describe the language in details here; the interested reader can refer to our works [1,3,9]; in Sect. 4, we report the simple query shown during the live demo and shortly explain the statements.

The aim of *J-CO-QL* is to provide a powerful tool to perform complex transformations on collections of *JSON* documents. More in details, the main features of the language are the following ones:

- *High-level Statements.* The language provides high-level statements, i.e., statements suitable to specify complex transformations without writing procedural programs;
- *Native Support to Spatial Operations.* The language natively manages geotagging in *JSON* documents and, consequently, its statements provide constructs for this purpose. In this demo, this feature is not highlighted, but the interested reader can refer to [1,9];
- *Managing Heterogeneous Documents. JSON* document stores are schema free, consequently, they can store heterogeneous *JSON* documents all together in the same collection. *J-CO-QL* statements are able to deal with heterogeneous documents in one single instruction. In this demo, this feature is not highlighted, but the interested reader can refer to [1,9];
- *Independence of the Data Source.* The language is tied neither to any data source nor to any query language provided by any data source. This way,

in principle, any data source could be accessed to get and save collections of *JSON* documents, because all processing activities are performed by the *J-CO-QL Engine*, with no intervention of *JSON* stores;
– *Simple and Clear Execution Semantics.* The language relies on a simple and clear execution semantics, based on the notion of *process state*. We briefly introduce it hereafter.

Execution Semantics. A *Process State* describes the current state of the execution of a query process. It is a tuple $s = \langle tc, IR, DBS \rangle$. Specifically, tc is called *temporary collection* and is the collection produced by the last executed instruction, which will be the input for the next instruction. IR is the *Intermediate-Result Database*, i.e., a process-specific database where to temporarily store collections that constitute intermediate results for the process. DBS is a set of *database descriptors* $dd = \langle url, dbms, name, alias \rangle$, whose fields, respectively, describe the connection URL, the specific DBMS (i.e., `"MongoDB"`, `"ElasticSearch"` and `"J-CO-DS"`), the name of the database and the alias used within the *J-CO-QL* query in place of its actual name.

A *query* q is a sequence (or *pipe*) of instructions $(i_1, i_2, , \ldots, i_n)$, where an instruction i_j is the application of a *J-CO-QL* statement.

The *initial process state* is $s_0 = \langle \emptyset, \emptyset, \emptyset \rangle$. Given an instruction $i_j \in q$, it can be seen as a function such that $i_j(s_{j-1}) = s_j$; in other words, an instruction (and the applied statement) can possibly change the temporary collection tc, the intermediate result database IR, and the database descriptors, that, in turn, can be used as input by the next instruction.

4 Demo: Cross-Analyzing Weather Measurements

We now show the main analysis task that was shown during the demo at the conference. Specifically, we showed how *J-CO-QL* can be used to cross-analyze weather measurements and rank them by means of user-defined fuzzy operators.

4.1 Dataset

All the investigated data have been downloaded from the *Open Data Portal of Regione Lombardia*, the administrative body of the region named Lombardia in northern Italy (https://dati.lombardia.it/). The data consist of measurements provided by atmospheric sensors that detect values related to temperatures, rain, wind speed and direction, solar radiation, humidity and others. Each sensor is specialized in providing one kind of measure (e.g., temperature) several times each day. The sensors are organized into weather stations located in various sites in Lombardia region. For this demo, we only considered temperature and rain data detected in a time period that covers one week from 1 March 2021 to 7 March 2021 in Bergamo area (see Fig. 2).

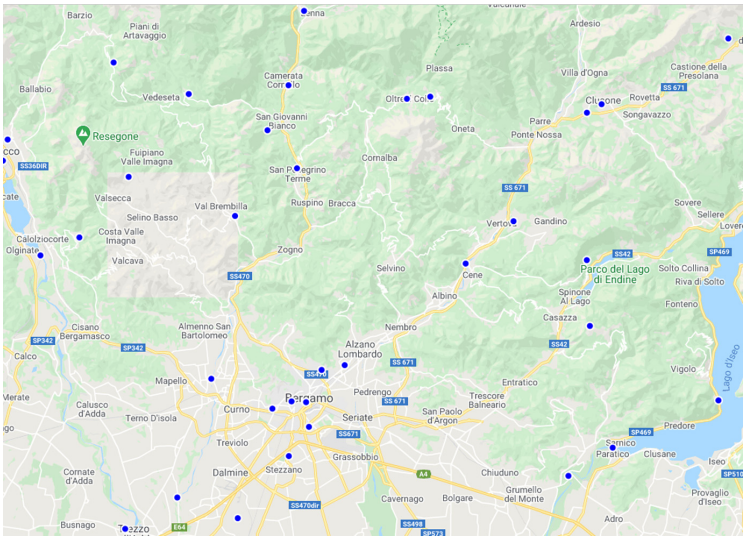


Fig. 2. Map of sensors.

The *Open Data Portal of Regione Lombardia*, provides the data in *JSON* format. Thus, we stored the downloaded data into a *MongoDB* database. Before conducting this experiment, all the data passed through a *pre-processing phase*, since stations and sensors data are provided together in a *de-normalized way*, meaning that while a station is a group of sensors in the same place and a sensor is a device able to measure a certain kind of physical quantity, the Open Data Portal provides a document for each sensor reporting in the same document also all the information about the station in which the sensor is located. Instead, measurement data are provided separately. Moreover, all field names are in Italian, so, they had to be translated into English. The pre-processing phase was performed by using a *J-CO-QL* script that we do not discuss here, since it is out of the scope of this paper and of the *live demo* during the conference. The queries submitted to the Open data portal to get the data are the following ones:

- `https://www.dati.lombardia.it/resource/nf78-nj6b.json`;
- `https://www.dati.lombardia.it/resource/647i-nhxk.json?$where=data >= "2021-03-01" AND data <"2021-03-08"`.

After the pre-processing phase, the data set contains two distinct collections of *JSON* documents:

- The `Temperatures` collection contains 26,215 documents. Each document represents a different temperature measurement in a precise moment. An example of document is shown in Fig. 3a;
- The `Rain` collection contains 32,362 documents. Each document represents a different rain measurement in a precise moment. Since the considered time

Listing 1. *J-CO-QL* cross-analysis process (part 1): Fuzzy operators.

```
1:  CREATE FUZZY OPERATOR Has_Medium_Temperature
        PARAMETERS temperature TYPE Float, altitude TYPE Integer
        PRECONDITION temperature >= -273 AND altitude >= 0
        EVALUATE temperature + 0.006*altitude
        POLYLINE (-30, 0), (0, 0), (5, 1), (15, 1), (25, 0), (50, 0);

2:  CREATE FUZZY OPERATOR Has_Persistent_Rain
        PARAMETERS rain TYPE Integer
        PRECONDITION rain > 0
        EVALUATE rain
        POLYLINE (0, 0), (0.5, 0), (1, 0.2), (1.5, 0.9), (1.7, 1), (2, 1);
```

period was quite dry in the area of interest, we discarded more than 99% of the documents that were reporting a *zero value* as a measurement. The choice was made also considering the need to speed-up the execution time during the live demo at the conference. Thus, the actual collection used for this demo contains only 238 meaningful documents. An example of document is shown in Fig. 3b.


```
{
    "stationId" : "595",
    "stationName" : "Filago v.Don Milani",
    "province" : "BG",
    "latitude" : "45.63386450908066",
    "longitude" : "9.556084255790864",
    "altitude" : "190",
    "sensorId" : "5863",
    "type" : "Temperature",
    "unit" : "°C",
    "dateTime" : "2021-03-07T23:00:00",
    "measure" : 3.1
}
```

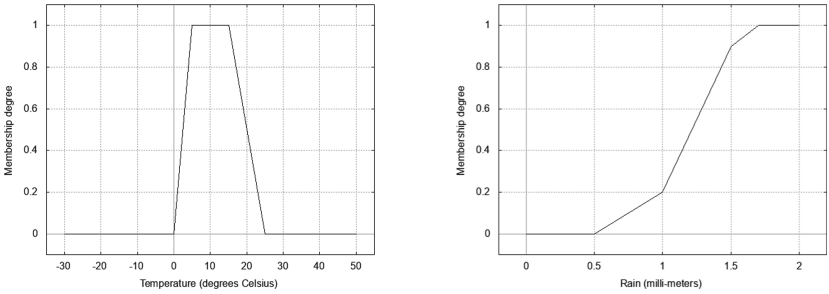

(a) `Temperatures` document.


```
{
    "stationId" : "821",
    "stationName" : "Azzone Dezzo di Scalve",
    "province" : "BG",
    "latitude" : "45.97808314682598",
    "longitude" : "10.103188671004808",
    "altitude" : "767",
    "sensorId" : "8171",
    "type" : "Rain Precipitation",
    "unit" : "mm",
    "dateTime" : "2021-03-06T08:10:00",
    "measure" : 0.2
}
```


(b) `Rain` document.

Fig. 3. Examples of documents in `Temperatures` and `Rain` collections.

In Fig. 3, the reader can see an example of documents contained respectively in the `Temperatures` collection and in the `Rain` collection. *JSON* documents in both collections have the same structure. Each document describes the weather station through its unique identifier and name (respectively, the `stationId` and the `stationName` fields), as well as the geographic area and the exact *WGS84* coordinates of the weather station are provided (respectively, the `province`, the `latitude`, the `longitude` and the `altitude` fields). Each document describes also the sensor through its unique identifier (the `sensorId` field), its type (the `type` field) and the unit of measure used for sensor measurements (the `unit` field). Finally, the exact date-time and value of measurement are reported (respectively, the `dateTime` and the `measure` fields).



(a) `Has_Medium_Temperature`. (b) `Has_Persistent_Rain`.

Fig. 4. Membership Functions of Fuzzy Operators in Listing 1

4.2 Creating Fuzzy Operators

The *J-CO-QL* language provides a specific construct to create novel fuzzy operators, so as to use them within queries to evaluate the extent of the belonging of *JSON* documents to fuzzy sets. Listing 1 reports the instructions that create two fuzzy operators, named `Has_Medium_Temperature` and `Has_Heavy_Rain`, respectively.

Let us consider the instruction that defines the `Has_Medium_Temperature` operator.

First of all, it defines the list of formal parameters necessary to the operator to provide a membership degree. Specifically, the operator receives two formal parameters, named `temperature` and `altitude`, which denote a measured temperature and the altitude of the measuring point, respectively.

The `PRECONDITION` clause specifies a precondition that must be met in order to correctly evaluate the membership degree. In the example, the temperature is considered valid if above -273 *degree Celsius* (that is, 0 *Kelvin*), as well as the altitude is considered valid if no less than 0 `meter` above the sea level.

The `EVALUATE` clause defines the function whose resulting value is used to get the actual membership degree. In the example, the temperature is compensated when the altitude increases (due to the altitude, temperatures become lower).

The value returned by the function specified in the `EVALUATE` clause is used as x-value; the corresponding y-value, given by the polyline function, is the final membership degree provided by the fuzzy operator.

The `POLYLINE` clause defines the polyline function depicted in Fig. 4a.

The second fuzzy operator defined in Listing 1 is the `Has_Persistent_Rain` operator. It receives only one parameter, whose value is directly used (see the simple function specified in the `EVALUATE` clause) to get the membership degree through the polyline depicted in Fig. 4b. Notice that the polyline is not a simple trapezoidal function: this way, we allow for specifying possibly complex membership functions to cope with complex real situations.

Listing 2. *J-CO-QL* cross-analysis process (part 2): Joining Collections.

```
3:  USE DB FQAS_Source
        ON SERVER MongoDB 'http://127.0.0.1:27017';
4:  USE DB FQAS_Destination
        ON SERVER MongoDB 'http://127.0.0.1:27017';

5:  JOIN OF COLLECTIONS
        Temperatures@FQAS_Source AS T, Rain@FQAS_Source AS R
        CASE WHERE .T.stationId = .R.stationId
                AND .T.dateTime = .R.dateTime
                AND .R.measure > 0
            GENERATE {  .stationId: .T.stationId,
                        .stationName: .T.stationName,
                        .province: .T.province,
                        .latitude: .T.latitude,
                        .longitude: .T.longitude,
                        .altitude: .T.altitude,
                        .dateTime: .T.dateTime,
                        .temperatureSensorId: .T.sensorId,
                        .temperatureUnit: .T.unit,
                        .temperatureMeasure: .T.measure,
                        .rainSensorId: .R.sensorId,
                        .rainUnit: .R.unit,
                        .rainMeasure: .R.measure  }
            DROP OTHERS;
```

4.3 J-CO-QL Cross-Analysis Process

The *J-CO-QL* instructions that create the two fuzzy operators (Listing 1) are the first part of the script that specifies the cross-analysis process. In Listing 2, we report the second part, which actually starts the analysis process.

On lines 3 and 4, we ask to connect to two databases managed by the *MongoDB* server installed on the PC to be used for the demo. They are called `FQAS_Source` and `FQAS_Destination`: the former contains the source data to analyze, the latter will contain the output data.

The `FQAS_Source` database contains the `Temperatures` collection, which describes measurements of temperatures, and the `Rain` collection, which describes measurements of rain, as described in Sect. 4.1. The `JOIN` instruction on line 5 keeps the two collections from the database and joins them, in order to pair temperature and rain measurements made by the same station in the same moment. The two collections are aliased as `T` and `R`, respectively.

Within the instruction, the `CASE WHERE` clause specifies the join condition: among all pairs of input documents, only those referring to the same station and to the same moment, as well as having a rain measurement greater than zero, are kept. For each pair of input documents, a novel document is built, where two fields named `T` and `R` (the collection aliases) are present: the former contains

the input document coming from the `T` collection, the latter contains the input document coming from the `R` collection. This motivates the *dot notation* used within the condition (e.g., `.T.stationId`, where the initial dot denotes that we refer to the root-level `T` field).

For each selected document, the `GENERATE` action restructures the output document; specifically, we flatten it. The output document is inserted into the output temporary collection, which will be used as input collection by the next instruction.

Notice the final `DROP OTHERS` option. It specifies that all documents obtained by pairing input documents that do not satisfy the condition must be discarded from the output temporary collection.

An example of document in the temporary collection is shown in Fig. 5a.


```
{
    "stationId" : "1221",
    "stationName" : "Casnigo Campo Sportivo",
    "province" : "BG",
    "latitude" : "45.811360742715465",
    "longitude" : "9.865590362771156",
    "altitude" : "502",
    "dateTime" : "2021-03-06T07:10:00.000",
    "temperatureSensorId" : "9024",
    "temperatureUnit" : "°C",
    "temperatureMeasure" : 4.7,
    "rainSensorId" : "9113",
    "rainUnit" : "mm",
    "rainMeasure" : 0.8
}
```


(a) Document after `JOIN` (line 5).


```
{
   "stationId" : "1209",
   "stationName" : "Rota d'Imagna",
   "province" : "BG",
   "latitude" : "45.839840095961094",
   "longitude" : "9.511348909108976",
   "altitude" : "674",
   "dateTime" : "2021-03-06T06:10:00",
   "temperatureSensorId" : "9034",
   "temperatureUnit" : "°C",
   "temperatureMeasure" : 4.2,
   "rainSensorId" : "9123",
   "rainUnit" : "mm",
   "rainMeasure" : 1.6,
   "~fuzzysets" : {
       "Medium_Temperature" : 0.84,
       "Persistent_Rain" : 0.949999976158142,
       "Wanted" : 0.84
   }
}
```


(b) Document after `FILTER` (line 8).

Fig. 5. Examples of documents in the temporary collection.

Listing 3 reports the fuzzy part of the cross-analysis process. It is based on four different applications of the `FILTER` statement, whose general goal is to filter the temporary collection. Since [10], the `FILTER` statement has been extended to provide constructs for performing fuzzy evaluations of documents.

On line 6, we evaluate the extent to which documents belong to the fuzzy set called `Medium_Temperature`. First of all, the `CASE WHERE` condition selects documents with the fields named `temperatureMeasure` and `altitude`. For those documents, the `CHECK FOR FUZZY SET` clause evaluates the desired fuzzy set, by using the fuzzy expression specified in the `USING` sub-clause, which provides the membership degree; in this case, the membership degree is provided by the `Has_Medium_Temperature` fuzzy operator, defined in Listing 1. As an effect, the output document is obtained by extending the input one with a novel field named `~fuzzysets`, which in turn contains a field named `Medium_Temperature`, whose numerical value is the membership degree of the document to the fuzzy set.

The `FILTER` instruction on line 7 evaluates the extent to which documents belong to the fuzzy set named `Persistent_Rain`, by using the operator named

Listing 3. *J-CO-QL* cross-analysis process (part 3): Evaluating Fuzzy Sets.

```
6:  FILTER
        CASE WHERE WITH .temperatureMeasure, .altitude
          CHECK FOR FUZZY SET Medium_Temperature
            USING Has_Medium_Temperature(.temperatureMeasure, .altitude)
        DROP OTHERS;

7:  FILTER
        CASE WHERE WITH .rainMeasure
          CHECK FOR FUZZY SET Persistent_Rain
            USING Has_Persistent_Rain(.rainMeasure)
        DROP OTHERS;

8:  FILTER
        CASE WHERE KNOWN FUZZY SETS Medium_Temperature, Persistent_Rain
          CHECK FOR FUZZY SET Wanted
            USING (Medium_Temperature AND Persistent_Rain)
            ALPHA-CUT 0.75 ON Wanted
        DROP OTHERS;

9:  FILTER
     CASE WHERE WITH .~fuzzysets.Wanted
          GENERATE {  .stationId, .stationName, .province, .dateTime,
                      .latitude, .longitude, .altitude,
                      .rank: MEMBERSHIP_OF (Wanted) }
          DROPPING ALL FUZZY SETS
        DROP OTHERS;

10: SAVE AS DetectedWeatherStations@FQAS_Destination;
```

`Has_Persistent_Rain`, defined in Listing 1. Within the `~fuzzysets` field, output documents now have a novel field, denoting the membership degree to the `Persistent_Rain` fuzzy set.

The `FILTER` instruction on line 8 combines the two previously evaluated fuzzy sets, to obtain the membership degree to the `Wanted` fuzzy set, whose value will be used to rank documents. In the `WHERE` condition, we use the predicate named `KNOWN FUZZY SETS`, which is true if the specified fuzzy sets have been evaluated for the current document. If so, the `Wanted` fuzzy set is evaluated, by means of the expression within the `USING` sub-clause: notice that it is based on the fuzzy `AND` conjunction applied to the previously checked fuzzy sets. In fact, the `WHERE` clause is a pure Boolean condition, while the `USING` clause is a fuzzy condition. Finally, the `ALPHA-CUT` sub-clause discards all documents whose membership degree to the `Wanted` fuzzy set is less than the specified threshold of `0.75`.

An example of document in the temporary collection, after the `FILTER` instruction on line 8, is shown in Fig. 5b. Notice the `~fuzzysets` field, whose

fields denotes the membership degrees of the document to the homonym fuzzy sets (i.e., `Medium_Temperature`, `Persistent_Rain` and `Wanted`).

The final `FILTER` instruction on line 9 prepares the final output collection of the process. In particular, the `rank` field is added, assigning it the membership degree of the `Wanted` fuzzy set by means of the `MEMBERSHIP_OF` built-in function. The `DROPPING ALL FUZZY SETS` option asks to drop fuzzy sets from the documents (i.e., the `~fuzzysets` field is removed).

Finally, the `SAVE AS` instruction at line 10 saves the final temporary collection into the `FQAS_Destination` database, with name `DetectedWeatherStations`. An example of document in this collection is shown in Fig. 6.


```
{
    "stationId" : "119",
    "stationName" : "San Giovanni Bianco C.",
    "province" : "BG",
    "dateTime" : "2021-03-06T05:10:00",
    "latitude" : "45.86969089938538",
    "longitude" : "9.639112837494569",
    "altitude" : "622",
    "rank" : 0.759999993443489
}
```


Fig. 6. Example of document in the `DetectedWeatherStation` collection.

5 Conclusions

During the live demo at FQAS 2021 Conference, we showed the *J-CO Framework* and the capability of its query language, called *J-CO-QL*, to retrieve and manipulate collections of *JSON* documents, by evaluating the extent of their belonging to fuzzy sets.

This paper accompanies the live demo, so as to describe it in the conference proceedings. Thus, it briefly presents the main characteristics of the *J-CO Framework* and its query language. The main goal of the framework is to provide analysts with a unifying tool to manage, query and transform big collections of *JSON* documents stored in *JSON* storage systems based on different technologies and provided with different query languages. The platform-independent design of the framework is the key factor to achieve this goal.

During the live demo, we presented a cross-analysis process of meteorological data, to show a typical, yet simplified, application of the *J-CO Framework*. The paper reports and explains the *J-CO-QL* query written to perform the cross-analysis; in particular, the way fuzzy sets are evaluated on *JSON* documents, based on the definition of fuzzy operators tailored on the application context.

The reader interested to learn more about the framework is kindly invited to read the related papers in bibliography and to get in touch with the authors. All the software developed within the project and the data used for this demo, and for other works, are freely accessible in the *GitHub* repository of the project (https://github.com/zunstraal/J-Co-Project/).

References

1. Bordogna, G., Capelli, S., Ciriello, D.E., Psaila, G.: A cross-analysis framework for multi-source volunteered, crowdsourced, and authoritative geographic information: the case study of volunteered personal traces analysis against transport network data. Geo-spat. Inf. Sci. **21**(3), 257–271 (2018)
2. Bordogna, G., Capelli, S., Psaila, G.: A big geo data query framework to correlate open data with social network geotagged posts. In: Bregt, A., Sarjakoski, T., van Lammeren, R., Rip, F. (eds.) GIScience 2017. LNGC, pp. 185–203. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-56759-4_11
3. Bordogna, G., Ciriello, D.E., Psaila, G.: A flexible framework to cross-analyze heterogeneous multi-source geo-referenced information: the J-CO-QL proposal and its implementation. In: Proceedings of the International Conference on Web Intelligence, pp. 499–508. ACM, Leipzig (2017)
4. Burini, F., Cortesi, N., Gotti, K., Psaila, G.: The urban nexus approach for analyzing mobility in the smart city: towards the identification of city users networking. Mob. Inf. Syst. **2018**, 1–18 (2018)
5. Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., Schaub, T., et al.: The GeoJSON format. Internet Engineering Task Force (IETF) (2016)
6. Fosci, P., Marrara, S., Psaila, G.: Soft querying GeoJSON documents within the J-CO framework. In: 16th International Conference on Web Information Systems and Technologies (WEBIST 2020), pp. 253–265. SCITEPRESS-Science and Technology Publications, Lda (2020)
7. Nayak, A., Poriya, A., Poojary, D.: Type of NOSQL databases and its comparison with relational databases. Int. J. Appl. Inf. Syst. **5**(4), 16–19 (2013)
8. Psaila, G., Fosci, P.: Toward an analyst-oriented polystore framework for processing JSON geo-data. In: IADIS International Conference Applied Computing 2018, pp. 213–222. IADIS, Budapest, Hungary (2018)
9. Psaila, G., Fosci, P.: J-CO: a platform-independent framework for managing geo-referenced JSON data sets. Electronics **10**(5) (2021). https://doi.org/10.3390/electronics10050621, https://www.mdpi.com/2079-9292/10/5/621
10. Psaila, G., Marrara, S.: A first step towards a fuzzy framework for analyzing collections of JSON documents. In: IADIS International Conference Applied Computing 2019, pp. 19–28. IADIS, Cagliari, Italy (2019)
11. Uddin, M.F., Gupta, N., et al.: Seven V's of big data understanding big data to extract value. In: Proceedings of the 2014 Zone 1 Conference of the American Society for Engineering Education, pp. 1–5. IEEE (2014)
12. Zadeh, L.A.: Fuzzy sets. Inf. Control **8**(3), 338–353 (1965)